Blood Donation Network

Weekly Mini Project 04 — Team 1 · Project Specification

Project	Blood Donation Network
Team leader	Iyad Ibrahim
Team members	Taima Watfeh, Adham Albasha, Mulham Al Kasir, Laith Haj Hosin, Yazan Shartouh
Stack	React + TypeScript · NestJS · PostgreSQL through Prisma · JWT · Swagger
Submission	Your instructor will confirm the submission date and the presentation date.

> When a patient needs blood urgently, find the right donors in the right area — fast, and without asking anyone who cannot donate yet.

1. Project Overview

When a hospital needs blood urgently, staff often post a message on social media and phone people one by one. Many of the people who answer have the wrong blood type, live too far away, or donated only a few weeks ago and are not yet allowed to donate again. Valuable time is lost.

Blood Donation Network is a coordination platform for emergency blood donation. A blood-bank coordinator publishes an SOS request (blood type, number of units, area, urgency). The system finds donors who are **active**, **compatible**, **in the same area** and **outside their rest period** after a previous donation. Donors can see open requests that they can actually help with and pledge to donate. When a donation is recorded, the request's unit count is updated and the donor automatically enters a rest period.

> **Important:** this application **coordinates and matches**. It is **not** a medical system. It does not decide whether a person is medically fit to donate and does not replace the screening that qualified medical staff perform before every donation. The compatibility table and rest period below are program rules used only to decide **whom to contact**.

2. Team

The team leader and members are listed in the table at the top of this document.

Team Responsibility

- The leader coordinates the internal work split and makes sure the team reaches a working integrated result.
- Frontend/backend ownership is not assigned by the instructor.
- The team should decide how responsibilities are distributed.
- Every member must make a meaningful contribution.
- Every member should understand the project at a high level.
- Every member must participate in the final English presentation.

3. Product Goals

By the end of the project, the application must allow:

- a coordinator to publish, update, cancel and track emergency blood requests;
- the system to produce a ranked list of eligible donors for any open request;

- donors to register, keep their profile and availability up to date, see the requests they can help with, and pledge to donate;
- a coordinator to record a completed donation, which updates the request's units and starts the donor's rest period in one safe operation;
- the public to see open SOS requests (without any personal data).

4. Technology Constraints

Area	Requirement
Frontend	React + TypeScript
Backend	NestJS
Database	PostgreSQL, accessed through Prisma (schema + migrations)
Authentication	JWT, with role checks on the backend
API documentation	Swagger / OpenAPI

- All data is stored in PostgreSQL and survives a backend restart.
- Every important rule is enforced by the backend. The frontend may also check input for a better experience, but the backend must reject invalid operations on its own.
- Do **not** add maps, external location services, SMS/email services, payment, real hospital systems, external AI services or cloud deployment.

5. User Roles

Role	Who	Can	Cannot
Coordinator	Blood-bank or hospital staff. Accounts are seeded; there is no public coordinator sign-up.	Create, update and cancel SOS requests; view eligible donors for a request; record completed donations; view all donors.	Edit a donor's personal profile; record a donation for a donor who is not eligible.
Donor	A person who signs up as a volunteer donor.	Manage their own profile, area and availability; see open requests that match them; pledge to one request; withdraw their pledge.	See other donors' personal data; create requests; record donations; pledge while ineligible.
Public visitor	Anyone, without signing in.	See the list of open SOS requests: blood type needed, area, urgency, units still needed.	See any donor's or patient's personal data.

6. User Scenario

At 09:10 a coordinator at the central blood bank publishes an SOS request: **A– blood, 3 units, area Homs, urgency CRITICAL**. The public SOS board now shows it.

The coordinator opens the request's **Eligible donors** list. It contains only donors who are active, available, in Homs, compatible with A– (that means A– and O– donors), and whose rest period has ended. Rami (O–) donated 20 days ago, so he is **not** in the list, and the request page explains that 1 compatible donor in the area is excluded because of the rest period.

Sara (A–, Homs) signs in. Her dashboard shows the CRITICAL request because she can help. She pledges. Her pledge appears on the request. At 11:30 she donates, and the coordinator records the donation: the request now shows **1 of 3 units collected**, Sara's pledge is marked completed, and Sara's profile shows she can donate again on a date 56 days later. She no longer appears in any eligible-donor list until then.

When the third unit is recorded, the request automatically becomes **FULFILLED**, disappears from the SOS board, and any remaining open pledges for it are released.

7. Main User Flows

Each flow is written as **user action → business operation → stored result → visible result**.

Flow A — Publish an SOS request

1. The coordinator fills in the request form → the backend validates it and checks there is no duplicate open request → a request with status **OPEN** and 0 collected units is stored → the request appears in the coordinator's list and on the public SOS board.

Flow B — Find eligible donors

1. The coordinator opens an open request and chooses **Eligible donors** → the backend applies every matching rule (section 9) → nothing is changed → a ranked list is shown, together with how many donors were excluded and why (rest period, inactive, unavailable).

Flow C — Donor pledges

1. A donor opens **Requests I can help with** → the backend returns only open requests the donor is eligible for → the donor pledges to one request → a **PLEDGED** pledge is stored → the request page shows the pledge, and the donor's dashboard shows their active pledge.
2. The donor can withdraw an active pledge → the pledge becomes **WITHDRAWN**.

Flow D — Record a donation

1. The coordinator records a donation for a donor on a request → in **one transaction** the backend creates the donation record, increases the request's collected units, completes the donor's pledge (if any), updates the donor's last donation date and, if the required units are reached, marks the request **FULFILLED** and releases other pledges → the request page, the donor list and the SOS board all show the new state.

Flow E — Cancel a request

1. The coordinator cancels an open request with a reason → the request becomes **CANCELLED** and all its open pledges are released → it disappears from the SOS board and from donors' lists.

8. Functional Requirements

Accounts and profiles

- **FR-1** A visitor can sign up as a donor with full name, phone number, password, blood type and area. The password is stored hashed.
- **FR-2** Coordinators and donors can sign in and receive a JWT. Coordinator accounts are seeded.
- **FR-3** A donor can view and update their own profile, set themselves **available / unavailable**, and deactivate their account.
- **FR-4** A donor sees their next allowed donation date and their donation history.

Requests

- **FR-5** A coordinator can create a request: patient reference (an internal code, not a name), blood type needed, units required (1–10), area, urgency (**CRITICAL**, **HIGH**, **NORMAL**), hospital or blood-bank name and an optional note.
- **FR-6** A coordinator can update an OPEN request's units required, urgency and note, and cancel an OPEN request with a reason.
- **FR-7** Coordinators can list requests and filter them by status, blood type, area and urgency.

- **FR-8** The public SOS board lists OPEN requests ordered by urgency (CRITICAL first) and then by creation time (oldest first), showing blood type, area, urgency and units still needed only.

Matching and pledges

- **FR-9** For any OPEN request, a coordinator can view the ranked list of eligible donors and a summary of excluded donors by reason.
- **FR-10** A donor can view the OPEN requests they are eligible for, ordered like the SOS board.
- **FR-11** A donor can pledge to an eligible OPEN request and withdraw their own active pledge.

Donations

- **FR-12** A coordinator can record a donation (donor, request, donation date) for an OPEN request.
- **FR-13** A request page shows units required, units collected, units still needed, pledges and recorded donations.

9. Business Rules

Blood-type compatibility (red cells)

- **R-1** A donor is **compatible** with a request when the donor's blood type appears in the request's row below. This table follows the standard red-cell compatibility published by national blood services (for example Australian Red Cross Lifeblood).

Blood type needed	Compatible donor blood types
O–	O–
O+	O+, O–
A–	A–, O–
A+	A+, A–, O+, O–
B–	B–, O–
B+	B+, B–, O+, O–
AB–	AB–, A–, B–, O–
AB+	all eight types

- **R-2** Compatibility must be decided by the backend from this table. The frontend never decides who is compatible.

Rest period after a donation

- **R-3** After a recorded donation, the application enforces a **configured rest period** (cooldown) before the donor can be matched again. In this project the configured value is **56 days**. It is an application rule used only to decide whom to contact — it is **not** a medical clearance. The value must be a single configuration setting, not repeated across the code.
- **R-4** A donor's **next allowed donation date** = last donation date + rest period. While today is before that date, the donor is **in rest period**.
- **R-5** A donor with no recorded donation has no rest period.

Eligibility for matching

- **R-6** A donor is **eligible** for a request only when **all** of these are true: the donor account is active; the donor is available; the donor is not in a rest period; the donor's blood type is compatible; the donor's area equals the request's area.

- **R-7** The eligible-donor list is ordered: exact blood-type match first, then other compatible types; within each group, the donor whose last donation is longest ago (or who never donated) comes first; then by name.
- **R-8** The excluded-donor summary counts compatible donors in the same area who are excluded, grouped by reason (rest period, unavailable, inactive). It never lists their personal data to the public.

Request lifecycle

- **R-9** A request is **OPEN**, **FULFILLED** or **CANCELLED**. Only OPEN requests can be updated, pledged to, matched or receive donations.
- **R-10 Units still needed** = units required − units collected. It is never negative.
- **R-11** When units collected reaches units required, the request becomes **FULFILLED** automatically.
- **R-12** Units required cannot be reduced below the units already collected.
- **R-13** Two OPEN requests with the same patient reference and the same blood type are not allowed.
- **R-14** FULFILLED and CANCELLED requests cannot be reopened.

Pledges

- **R-15** A donor can pledge only to an OPEN request they are eligible for.
- **R-16** A donor can hold **at most one active (PLEDGED) pledge** at a time, across all requests.
- **R-17** A request cannot have more active pledges than its units still needed.
- **R-18** When a request becomes FULFILLED or CANCELLED, its remaining PLEDGED pledges become **RELEASED**.

Recording a donation

- **R-19** A donation can be recorded only for an OPEN request and only for a donor who is eligible for it at the moment of recording.
- **R-20** One recorded donation counts as **one unit**.
- **R-21** The donation date cannot be in the future.
- **R-22** Recording a donation, increasing units collected, completing the donor's pledge, updating the donor's last donation date and (when reached) fulfilling the request and releasing other pledges must happen **in one database transaction**: either everything is saved, or nothing is.

Privacy

- **R-23** Public endpoints and the SOS board never return donor names, phone numbers, patient references or notes.
- **R-24** A donor can read and change only their own profile, pledges and history.

10. Frontend Requirements

Required screens

Screen	Who	Must show / allow
SOS Board	Everyone	OPEN requests with blood type, area, urgency badge and units still needed; empty state when there are none.
Sign in / Sign up	Everyone	Sign-in for both roles; donor sign-up form with validation messages.
Donor Dashboard	Donor	Eligibility status (eligible / in rest period until date / unavailable), next allowed donation date, active pledge, requests I can help with, donation history.
Donor Profile	Donor	Edit area, phone and availability; deactivate account.
Requests	Coordinator	All requests with filters; create request action.

Screen	Who	Must show / allow
Request Details	Coordinator	All request data, units required/collected/still needed, pledges, donations, update/cancel actions, record-donation action.
Eligible Donors	Coordinator	Ranked eligible donors with blood type, area and last donation date; excluded-donor summary by reason.

Required behavior

- Every screen that loads data shows a **loading** state, an **empty** state when there is nothing to show, and an **error** state when a request fails.
- Forms show the backend's validation messages next to the form, not only in the console.
- Business-rule rejections (for example "this donor is in rest period until 12 Dec 2026") are shown clearly to the user.
- After a successful action, the affected lists and counters update without a manual page reload.
- When there is no valid sign-in, screens that need one show a sign-in prompt instead of their content.

11. Data Modeling Guidance

The information below must be represented. How you split it into Prisma models, fields and relations is your team's design decision.

Entity	Purpose	Important information	Important relationships
User account	Signing in and role	email or phone, hashed password, role, active flag	One donor profile per donor account
Donor profile	The donor's matching data	full name, blood type, area, available flag, last donation date	Belongs to one user; has many pledges and donations
Blood request	An SOS request	patient reference, blood type, units required, units collected, area, urgency, status, hospital, note, cancel reason, created/updated times	Created by a coordinator; has many pledges and donations
Pledge	A donor's promise to donate	status (PLEDGED, COMPLETED, WITHDRAWN, RELEASED), created time	One donor, one request
Donation	A completed donation	donation date, units (always 1), recorded by	One donor, one request, one coordinator
Area	The fixed list of supported areas	name	Used by donors and requests

Keep **required domain data** (above) separate from your **implementation choices**. For example, whether *units collected* is stored on the request or counted from donations is your decision — explain it in your presentation.

12. Backend Requirements

- **Architecture:** Controller → Service → Persistence (Prisma). Controllers receive requests and return responses; services contain all business rules; database access happens only in services. No database queries and no business rules in controllers.
- **Validation:** every request body, route parameter and query parameter is validated with DTOs; unknown fields are rejected.
- **Authentication and roles:** JWT sign-in; coordinator-only and donor-only operations are protected on the backend with guards. Public endpoints are deliberately open and return public fields only.

- **HTTP errors:** use the correct kind of error — bad request, unauthorized, forbidden, not found, conflict — with a clear message.
- **Transactions:** recording a donation (R-22) uses a database transaction.
- **Filtering:** request listing supports the filters in FR-7.
- **Logger:** log meaningful events — request created, cancelled and fulfilled, donation recorded, and rejected donation attempts with the reason. Do not log every request, and never log passwords or tokens.
- **Swagger:** every endpoint documented with a summary, grouped by resource, with bodies, parameters and the required authentication visible.
- **Persistence:** all data survives a restart; a seed script creates the areas, at least 2 coordinators, at least 20 donors covering all eight blood types (some in rest period, some unavailable), and several requests in different states.

13. Validation & Error Scenarios

Situation	Expected behavior
Sign-up with a missing field, an unknown blood type or an unknown area	Rejected with a clear message.
Sign-up with a phone number or email that is already registered	Rejected as a conflict.
Sign-in with a wrong password	Rejected as unauthorized, without saying which part was wrong.
A donor calls a coordinator-only operation	Rejected as forbidden.
Any protected operation without a valid token	Rejected as unauthorized.
Units required outside 1–10, or not a whole number	Rejected.
A second OPEN request with the same patient reference and blood type	Rejected as a conflict.
Reducing units required below units collected	Rejected.
Updating, cancelling, pledging to or recording a donation for a FULFILLED or CANCELLED request	Rejected as a conflict.
A donor pledges while in rest period, unavailable, incompatible or in another area	Rejected with the exact reason.
A donor with an active pledge pledges again	Rejected as a conflict.
A pledge when active pledges already equal units still needed	Rejected as a conflict.
Recording a donation for an ineligible donor, or with a future date	Rejected with the reason; nothing is saved.
A request or donor that does not exist	"Not found".
The SOS board when there are no open requests	Empty state, not an error.

14. CORE

Everything in this list is required for a complete submission:

- Sign-up, sign-in with JWT, coordinator/donor roles enforced on the backend.
- Donor profile, availability, deactivation, next allowed donation date, donation history.
- Request create / update / cancel / list with filters; public SOS board.
- Compatibility table, rest period, eligibility rules and the ranked eligible-donor list with the excluded summary.
- Pledges with all pledge rules.
- Recording donations in a transaction, automatic FULFILLED status and pledge release.

- All screens in section 10 with loading, empty and error states.
- Validation, HTTP errors, Logger, Swagger and seed data as described in section 12.

15. STRETCH (optional)

- **Request expiry:** a "needed by" date; OPEN requests past it are shown as overdue.
- **Coordinator statistics:** requests fulfilled this month, average time to fulfil, units collected per blood type.
- **Pledge reminder view:** donors with pledges older than 24 hours and no recorded donation.
- **Donor search for coordinators:** search by name or phone with filters by blood type and area.

16. CHALLENGE (optional, harder)

- **Distance-based matching:** store latitude/longitude for donors and requests and match donors within a chosen radius (calculated in your own code, without any maps service), still respecting every other rule.
- **Shortage forecast:** for each blood type and area, compare open demand with the number of currently eligible donors and highlight shortages.

17. OUT OF SCOPE

Do not build: medical diagnosis or medical fitness checks; laboratory or blood-testing systems; a hospital or blood-bank ERP; blood-unit inventory, storage or expiry tracking; ambulance dispatch; real emergency-service integration; SMS, email or push notifications; maps or external location services; payments; real-time WebSockets; mobile apps; cloud deployment.

18. Presentation Requirements

Your team presents **in English**. Every member speaks, and **any team member may be asked to explain any part of the project** — the presentation is not evaluated on the leader alone.

Your presentation must show:

1. the problem and the product idea;
2. the main user flow (publish request → eligible donors → pledge → record donation → fulfilled);
3. frontend behavior, including loading, empty and error states;
4. backend/API behavior in Swagger;
5. the persisted data in PostgreSQL (for example with Prisma Studio or SQL);
6. at least one meaningful business rule, explained in your own words (for example compatibility or the rest period);
7. a validation or error case being rejected;
8. your architecture and code organization (controller → service → persistence);
9. one technical or product decision your team made, and why;
10. one limitation of your product, or the next improvement you would make.

19. Acceptance Checklist

- □ A donor can sign up, sign in, and see their eligibility status and next allowed donation date.
- □ A donor calling a coordinator-only endpoint receives a forbidden error.
- □ A new request appears on the public SOS board, which shows no personal data.
- □ The eligible-donor list for an A– request contains only A– and O– donors from the same area, outside rest period, available and active, in the required order.

- □ The excluded-donor summary counts donors excluded for rest period, unavailability and inactivity.
- □ A donor in rest period cannot pledge, and the message gives the next allowed date.
- □ A donor with an active pledge cannot pledge again.
- □ Recording a donation increases units collected by 1, completes the pledge and starts the donor's configured rest period (56 days in this project).
- □ Recording the last needed unit marks the request FULFILLED and releases remaining pledges.
- □ If any step of recording a donation fails, nothing is saved (demonstrated or explained with the transaction code).
- □ FULFILLED and CANCELLED requests reject updates, pledges and donations.
- □ Data is still there after restarting the backend.
- □ Swagger documents every endpoint and its authentication.
- □ Meaningful log lines appear when a donation is recorded and when one is rejected.

20. Evaluation Focus

Area	What we look for
Functional completeness	The CORE features and screens exist and work.
Correct business behavior	Compatibility, rest period, eligibility, pledges and request lifecycle follow the rules exactly.
Backend/API quality	Controller → service → persistence, DTO validation, guards, correct HTTP errors, Swagger, Logger.
Database and persistence	A sensible relational model, migrations, seed data, the donation transaction.
Frontend quality	Clear screens with loading, empty, error and business-rule feedback.
Team collaboration and Git	Shared work, visible contributions from every member.
Explaining your own work	Every member can explain the product, its rules and its design.
Presentation quality	Clear English presentation with a working live demo.